
COMPUTER FUNDAMENTALS & PROGRAMMING IN C

Handwritten-style Study Notes

Computer System • Memory • Operating System & Applications •
Internet

Problem Solving • Algorithms & Flowcharts • C Language Basics

*Prepared with worked examples, labelled diagrams,
and complete C programs with sample output.*

TABLE OF CONTENTS

Unit 1	The Computer System — Definition, Block Diagram, Working, Peripherals, Hardware & Software
Unit 2	Computer Memory — Basic Concepts and Types of Memory
Unit 3	Operating System, Application Software & the Internet
Unit 4	Problem Solving — Algorithms, Flowcharts and Pseudocode
Unit 5	Introduction to the C Language — Structure, Preprocessor, History
Unit 6	Data Types and Operators in C

UNIT 1

The Computer System

1.1 What is a Computer System?

DEFINITION — Computer

A computer is an electronic device that accepts data (raw facts and figures) as input, processes that data by performing arithmetic and logical operations according to a set of stored instructions (a program), and produces meaningful information as output. It also stores data and instructions for future use.

A computer system is not just the machine (hardware) — it is the complete, coordinated set of hardware, software, data, procedures and people that work together to convert input into useful output. Every computer system, big or small, is built around one common cycle:

Input → Process → Output, together with Storage for keeping data and results for later use. This is called the IPOS cycle.

1.2 Block Diagram of a Computer System

A computer system is organised into five functional units. The block diagram below shows how these units are connected and how data and control signals flow between them.

Block Diagram of a Computer System

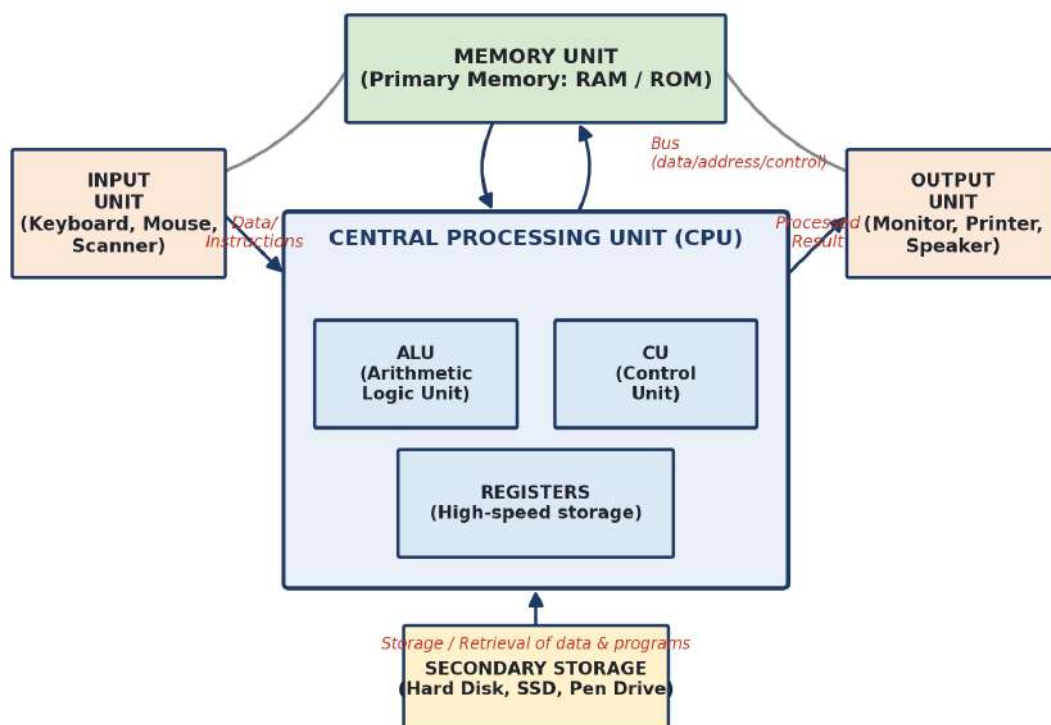


Fig. 1.1 – Block diagram showing the five functional units of a computer

The Five Functional Units

Unit	Function
Input Unit	Accepts data and instructions from the outside world (user) and converts them into a machine-readable (binary) form. Example devices: keyboard, mouse, scanner.
Central Processing Unit (CPU)	The 'brain' of the computer. It fetches instructions from memory, decodes them, and executes them. It consists of the ALU, the CU and Registers.
– Arithmetic Logic Unit (ALU)	Performs all arithmetic operations (+, -, ×, ÷) and logical/relational operations (AND, OR, NOT, >, <, =).
– Control Unit (CU)	Directs and coordinates the operations of the entire computer. It fetches instructions, decodes them and generates control signals to execute them, without doing the actual data processing itself.
– Registers	Very small, extremely fast storage locations inside the CPU used to hold data, addresses and intermediate results during processing (e.g. Accumulator, Program Counter).
Memory (Storage) Unit	Stores data, instructions and results – both before processing (input) and after processing (output). Divided into Primary and Secondary memory (see Unit 2).
Output Unit	Converts the machine's processed (binary) results back into a human-readable form. Example devices: monitor, printer, speaker.

1.3 Working of a Computer System

A computer works in a continuous cycle known as the Input–Process–Output–Storage (IPOS) cycle:

- Input: Data and instructions are entered through an input device (e.g. typing on a keyboard). The input unit converts this into binary form (0s and 1s), the only language the CPU understands.
- Storage (before processing): The binary data is placed in primary memory (RAM) so the CPU can access it quickly.
- Processing: The Control Unit fetches each instruction from memory, decodes what it means, and directs the ALU to perform the required arithmetic/logical operation. This basic cycle is called the Fetch–Decode–Execute cycle.
- Storage (after processing): The result of processing is stored back in memory.
- Output: The processed result is sent to an output device and converted into a human-readable form (text, image, sound, print-out).

Fetch–Decode–Execute Cycle (in short)

FETCH the instruction from memory -> DECODE it to find out what operation is required -> EXECUTE it using the ALU/CU -> STORE the result. This cycle repeats, at very high speed (millions/billions of times per second), for every single instruction in a program.

1.4 Peripheral Devices Associated with a Computer

A peripheral device is any hardware component that is connected to the computer externally and is used to feed data in or take results out; it is not part of the core CPU/memory. Peripherals are classified as follows:

Category	Examples
Input Peripherals	Keyboard, Mouse, Joystick, Scanner, Microphone, Light Pen, Touch Screen, Web Camera, Barcode Reader
Output Peripherals	Monitor (VDU), Printer (Dot-matrix/Inkjet/Laser), Plotter, Speaker, Projector
Storage Peripherals	Hard Disk Drive (HDD), Solid State Drive (SSD), Pen Drive/USB, CD/DVD, Memory Card
Communication Peripherals	Modem, Network Interface Card (NIC), Router

1.5 Computer Hardware and Software

DEFINITION – Hardware

Hardware refers to all the physical, tangible parts of a computer system that can be seen and touched – such as the CPU, monitor, keyboard, mouse, hard disk and printer.

DEFINITION – Software

Software is a set of programs, procedures and related data that provide instructions telling the hardware what to do and how to do it. Software cannot be touched, only executed/used.

Types of Software

Type	Description & Examples
System Software	Manages and controls the hardware so that application software can run. Examples: Operating Systems (Windows, Linux, macOS), Device Drivers, Language Translators (Compiler, Interpreter, Assembler).
Application Software	Designed to help the user perform specific tasks. Examples: MS Word (word processing), MS Excel (spreadsheets), MS PowerPoint (presentations), web browsers, games.
Utility Software	Performs maintenance-type tasks for the system. Examples: Antivirus, Disk Defragmenter, File compression tools (WinRAR).

Hardware vs. Software — At a Glance

Basis	Hardware	Software
Nature	Physical, tangible	Logical, intangible
Can be seen/touched?	Yes	No
Manufactured	In a factory	Developed by programmers
Gets damaged by	Dust, heat, physical wear	Viruses, bugs, logical errors
Examples	CPU, RAM, Keyboard, Monitor	Windows OS, MS Word, Chrome

Remember

Hardware and software are interdependent — hardware without software is just an idle machine, and software cannot run without hardware to execute it.

UNIT 2

Computer Memory

2.1 Basic Concepts of Memory

DEFINITION — Memory

Memory refers to the storage space in a computer where data and instructions are stored, either temporarily or permanently, so that the CPU can access and use them for processing.

The smallest unit of memory is a bit (binary digit — either 0 or 1). Memory is measured in the following units, each 1024 times larger than the previous:

Unit	Size
Bit	Smallest unit; a single binary value (0 or 1)
Nibble	4 bits
Byte	8 bits — enough to store one character
Kilobyte (KB)	1024 Bytes
Megabyte (MB)	1024 KB
Gigabyte (GB)	1024 MB
Terabyte (TB)	1024 GB

Key characteristics used to describe any memory:

- Capacity — the total amount of data it can hold.
- Access time — the time taken to read/write data.
- Volatility — whether data is lost when power is switched off (volatile) or retained (non-volatile).
- Cost per bit — faster memory is generally more expensive per bit stored.

2.2 Types of Memory

Computer memory is organised as a hierarchy: memory closest to the CPU is the fastest, smallest and most expensive, while memory farthest away is the slowest, largest and cheapest.

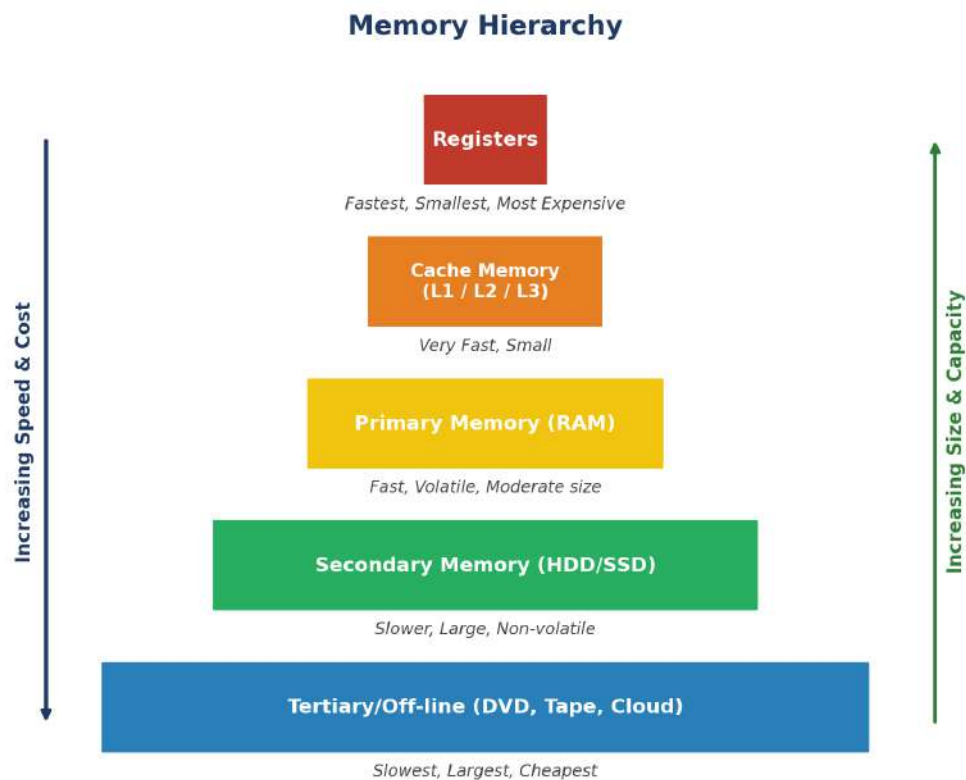


Fig. 2.1 – The Memory Hierarchy

(a) Primary Memory (Main Memory)

Directly accessible by the CPU; temporary/semi-permanent in nature; smaller in size but very fast.

Type	Description	Sub-types
RAM (Random Access Memory)	Volatile – read/write memory used to hold the program and data currently being processed. Lost when power is switched off.	SRAM (Static RAM – faster, used in cache) & DRAM (Dynamic RAM – needs refreshing, used as main memory)
ROM (Read Only Memory)	Non-volatile – permanently stores essential start-up instructions (e.g. BIOS) that cannot normally be modified by the user.	PROM, EPROM, EEPROM

(b) Cache Memory

A very small, very fast memory placed between the CPU and RAM. It stores copies of frequently used data/instructions so the CPU does not have to wait for the (slower) RAM every time – this greatly speeds up processing. Levels: L1 (fastest, smallest, built into the CPU core), L2 and L3 (progressively larger and slightly slower).

(c) Secondary Memory (Auxiliary/External Storage)

Non-volatile, larger in capacity, slower, and cheaper than primary memory. Used for permanent/long-term storage of data and programs even when the power is off.

- Hard Disk Drive (HDD) – magnetic storage, large capacity, moderate speed.

- Solid State Drive (SSD) — flash-memory based, much faster than HDD, no moving parts.
- Optical Disks — CD, DVD, Blu-ray — use laser light to read/write data.
- Flash/USB Drives & Memory Cards — portable, solid-state storage.

(d) Registers

The fastest memory of all, located inside the CPU itself. Registers hold data that the ALU is currently working on. Very small in number and size (a few bytes), but essential for speed.

Primary vs. Secondary Memory — Comparison

Basis	Primary Memory	Secondary Memory
Speed	Very fast	Slower
Cost per byte	Higher	Lower
Capacity	Smaller (GB range)	Larger (TB range)
Volatility	Usually volatile (RAM)	Non-volatile
CPU access	Direct	Indirect (via primary memory)
Examples	RAM, ROM, Cache	HDD, SSD, DVD, Pen Drive

Quick Tip

Remember the order of speed (fastest to slowest): Registers > Cache > RAM > SSD > HDD > DVD/Tape.

UNIT 3

Operating System, Applications & Internet

3.1 Introduction to Operating System (OS)

DEFINITION — Operating System

An Operating System is system software that acts as an intermediary between the computer user and the computer hardware. It manages hardware resources (CPU, memory, storage, devices) and provides common services for application programs, so that the user does not need to deal with hardware directly.

Functions of an Operating System

- Process Management — creating, scheduling and terminating processes/programs.
- Memory Management — allocating and freeing memory space to programs as needed.
- File Management — organising, storing, naming, and retrieving files and folders.
- Device Management — controlling input/output devices through drivers.
- Security & Access Control — protecting data through passwords and permissions.
- User Interface — provides a Command Line Interface (CLI) or Graphical User Interface (GUI) so users can interact with the system.

Examples of Operating Systems

- Windows (10, 11) — most common GUI-based OS for personal computers.
- Linux — open-source, highly customizable OS, widely used on servers.
- macOS — used on Apple Macintosh computers.
- Android / iOS — operating systems for mobile devices.

3.2 Word Editing Systems (Word Processing)

DEFINITION — Word Processor

A word processor is application software used to create, edit, format, save and print text documents, e.g. Microsoft Word, WPS Writer, Google Docs, LibreOffice Writer.

Common Features

- Text editing — insert, delete, copy, cut, paste, undo/redo.
- Formatting — font style/size/colour, bold, italic, underline, paragraph alignment, line spacing.
- Page layout — margins, page size/orientation, headers and footers, page numbers.

- Proofing tools — spelling & grammar check, thesaurus, word count.
- Objects — insertion of tables, pictures, shapes, charts, hyperlinks, headers/footers.
- Mail Merge — creating multiple personalised letters from one template and a data source.
- Saving/Exporting — saving in formats like .docx, .pdf, .txt, .rtf.

3.3 Creating PowerPoint Presentations

DEFINITION — Presentation Software

Presentation software (e.g. Microsoft PowerPoint, Google Slides) is used to create a sequence of slides that combine text, images, charts, audio and video to visually communicate ideas to an audience.

Basic Steps to Create a Presentation

1. Open PowerPoint and choose a blank presentation or a design template/theme.
2. Insert a new slide (Home -> New Slide) and choose a suitable slide layout (Title Slide, Title & Content, Two Content, etc.).
3. Add content — type the title and body text, insert images, SmartArt, charts or tables.
4. Apply a consistent Design/Theme for uniform colours and fonts across all slides.
5. Add Transitions (effects between slides) and Animations (effects on individual objects) if desired.
6. Use the Slide Sorter view to rearrange slide order.
7. Run Slide Show (F5) to preview, then Save the file (.pptx).

Key Terms

- Slide — a single page of a presentation.
- Slide Layout — the arrangement of placeholders (title, text, image) on a slide.
- Slide Master — a master template controlling the look of every slide.
- Transition — the visual effect when moving from one slide to the next.
- Animation — effects applied to text/objects within a single slide.

3.4 Creating Spreadsheets

DEFINITION — Spreadsheet Software

Spreadsheet software (e.g. Microsoft Excel, Google Sheets, LibreOffice Calc) organises data into a grid of rows and columns (called a worksheet), enabling calculations, analysis and visual presentation of numeric and textual data.

Basic Concepts

- Workbook — an Excel file, which can contain multiple worksheets.

- Worksheet — a single grid/sheet made of rows (numbered) and columns (lettered).
- Cell — the intersection of a row and column, identified by a cell reference (e.g. B4).
- Formula — an expression starting with '=' used to perform a calculation, e.g. =A1+A2.
- Function — a built-in formula, e.g. =SUM(A1:A10), =AVERAGE(B1:B5), =IF(A1>50, "Pass", "Fail").
- Cell Referencing — Relative (A1), Absolute (\$A\$1), and Mixed (\$A1 or A\$1) references.

3.5 Creating Simple Graphs/Charts

Spreadsheet software allows numeric data to be turned into visual charts for easier interpretation. General steps:

1. Select the data range (including row/column headings) that you want to chart.
2. Go to Insert -> Charts and choose a chart type (Column, Bar, Line, Pie, Scatter, etc.).
3. The chart is inserted on the worksheet; use Chart Design/Format tools to add a title, axis labels, legend and data labels.
4. Resize/move the chart, and save the worksheet.

Chart Type	Best Used For
Column/Bar Chart	Comparing values across categories
Line Chart	Showing a trend over a continuous period (e.g. time)
Pie Chart	Showing proportion/percentage of a whole
Scatter Chart	Showing the relationship/correlation between two numeric variables

3.6 Evolution of the Internet

DEFINITION — Internet

The Internet is a vast, global network of interconnected computer networks that communicate using a common set of protocols (TCP/IP), allowing computers all over the world to exchange data and information.

Period	Milestone
1969	ARPANET, developed by the U.S. Department of Defense (ARPA), goes online — the first working packet-switched network and the ancestor of the Internet.
1970s	Development of TCP/IP (Transmission Control Protocol/Internet Protocol) as the standard communication protocol.
1983	ARPANET officially adopts TCP/IP; this date is often marked as the 'birth' of the modern Internet.
1989–91	Tim Berners-Lee invents the World Wide Web (WWW) — a system of interlinked hypertext documents accessed via the Internet.

Period	Milestone
1990s	Web browsers (Mosaic, Netscape) and dial-up connections bring the Internet to homes and businesses; commercial use begins.
2000s onward	Broadband, Wi-Fi, mobile Internet (3G/4G/5G), social media, cloud computing and e-commerce transform the Internet into an essential part of daily life.

3.7 Internet Applications and Services

Service	Description
World Wide Web (WWW)	A collection of linked web pages/documents accessed through a browser using HTTP/HTTPS.
Electronic Mail (E-mail)	Sending and receiving digital letters/messages, e.g. Gmail, Outlook.
Search Engines	Tools to locate information on the web, e.g. Google, Bing.
Social Networking	Platforms for connecting and sharing, e.g. Facebook, Instagram, LinkedIn.
File Transfer Protocol (FTP)	Transferring files between computers over a network.
Instant Messaging & VoIP	Real-time text/voice/video chat, e.g. WhatsApp, Skype, Zoom.
E-commerce	Online buying and selling of goods/services, e.g. Amazon, Flipkart.
Cloud Computing/Storage	Using remote servers to store data and run applications, e.g. Google Drive, Dropbox.
E-learning	Online education and training platforms, e.g. Coursera, SWAYAM.

UNIT 4

Problem Solving Techniques

4.1 Need for Problem Solving

Before writing any computer program, a programmer must clearly understand the problem and plan a logical solution. Jumping straight into code without a plan usually leads to programs that are buggy, inefficient, or that solve the wrong problem. A systematic problem-solving approach:

- Ensures the exact requirements of the problem are understood before coding begins.
- Helps break a large, complex problem into smaller, manageable steps.
- Makes the logic language-independent, so it can later be implemented in any programming language.
- Reduces errors, saves development time, and makes the program easier to test and maintain.

Steps in Problem Solving

1. Analyse and understand the problem (identify inputs, outputs, and constraints).
2. Design a solution/algorithm to solve the problem.
3. Represent the solution using a flowchart or pseudocode.
4. Code the algorithm in a programming language (e.g. C).
5. Test and debug the program using sample data.
6. Document and maintain the program.

4.2 Program Planning

Program planning is the phase where the programmer decides what the program should do and how it will do it, before any actual coding starts. Good planning identifies:

- Inputs – what data the program needs and where it comes from.
- Outputs – what result the program must produce.
- Processing steps – the sequence of operations that convert input into output.
- Logic/control flow – decisions (if/else) and repetitions (loops) required.

4.3 Program Design Tools

Program design tools are used to plan the logic of a program in a way that is easy to understand, verify and later convert into actual code. The most common tools are:

Tool	Description
Algorithm	A step-by-step, ordered set of instructions in plain English to solve a problem.
Flowchart	A pictorial/graphical representation of an algorithm using standard symbols connected by arrows.
Pseudocode	A structured, code-like description of an algorithm written in plain language with programming-style keywords (IF, WHILE, etc.), independent of any specific language's syntax.
Decision Table	A table listing all possible conditions and the corresponding actions to be taken.

4.4 Algorithms

DEFINITION — Algorithm

An algorithm is a finite, well-defined sequence of unambiguous steps that takes some input, processes it, and produces the desired output in a finite amount of time.

Characteristics of a Good Algorithm

- Finiteness — it must terminate after a finite number of steps.
- Definiteness — every step must be precisely and unambiguously defined.
- Input — it should have zero or more well-defined inputs.
- Output — it must produce at least one well-defined output.
- Effectiveness — every step must be basic enough to be carried out, in principle, exactly and in finite time.

Illustrative Example 1: Algorithm to find the largest of two numbers

```

Step 1: Start
Step 2: Read two numbers A and B
Step 3: If A > B then
        Print "A is the largest number"
    Else
        Print "B is the largest number"
Step 4: Stop

```

Illustrative Example 2: Algorithm to find the sum of first N natural numbers

```

Step 1: Start
Step 2: Read the value of N
Step 3: Set SUM = 0 and I = 1
Step 4: Repeat steps 5 and 6 while I <= N
Step 5:     SUM = SUM + I
Step 6:     I = I + 1
Step 7: Print SUM
Step 8: Stop

```

4.5 Flowcharts

DEFINITION — Flowchart

A flowchart is a diagrammatic representation of an algorithm, in which standard geometrical symbols represent different types of operations, and arrows show the sequence/flow of control between them.

Common Flowchart Symbols

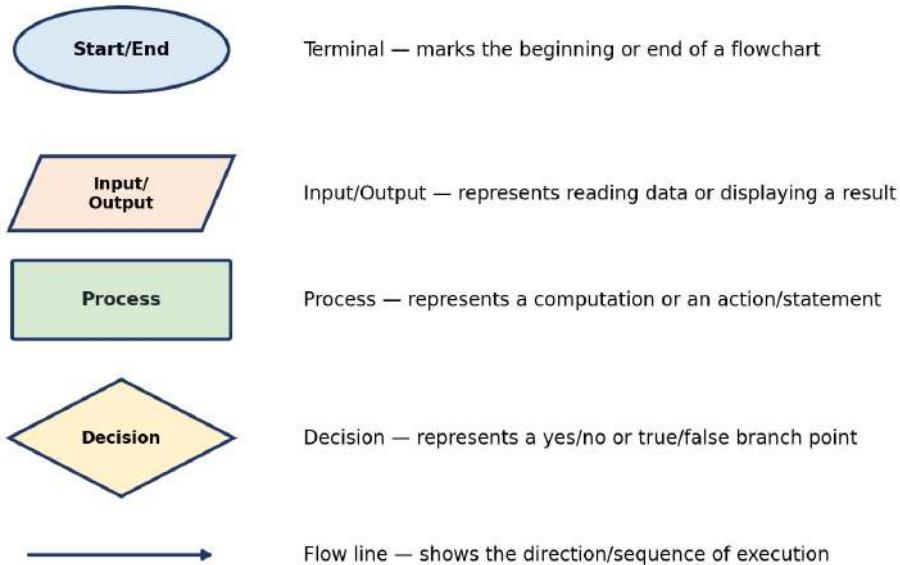


Fig. 4.1 — Standard flowchart symbols

Advantages of Flowcharts

- Provide a clear, visual picture of the program logic that is easy to understand.
- Serve as a blueprint during the program-coding phase.
- Act as a means of communication between the programmer and other stakeholders.
- Useful for debugging and later maintenance of the program.

Illustrative Example: Flowchart to find the largest of two numbers

Flowchart: Largest of Two Numbers

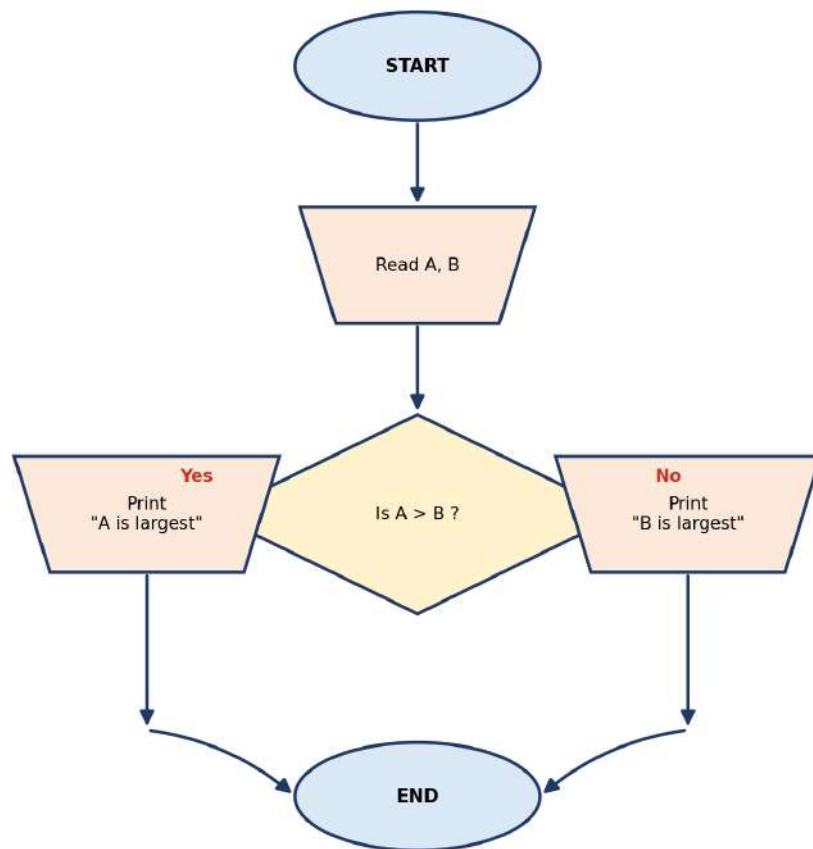


Fig. 4.2 — Flowchart for finding the largest of two numbers

4.6 Pseudocode

DEFINITION — Pseudocode

Pseudocode is an informal, structured way of writing an algorithm using a mixture of plain English and simple programming-style keywords (e.g. BEGIN, END, IF...THEN...ELSE, WHILE, FOR, READ, WRITE). It has no strict syntax rules and is not tied to any particular programming language, which makes it quick to write and easy for anyone to understand.

Illustrative Example: Pseudocode to check whether a number is even or odd

```
BEGIN
  READ number
  IF (number MOD 2 == 0) THEN
    PRINT "The number is Even"
  ELSE
    PRINT "The number is Odd"
  END IF
END
```

4.7 More Illustrative Examples (Algorithm + Pseudocode)

Example A: Simple Interest = $(P \times R \times T) / 100$

Algorithm

Step 1: Start

Step 2: Read P, R, T

Step 3: $SI = (P \times R \times T) / 100$

Step 4: Print SI

Step 5: Stop

Pseudocode

BEGIN

 READ P, R, T

$SI = (P \times R \times T) / 100$

 PRINT SI

END

Example B: Find the factorial of a number N

Step 1: Start

Step 2: Read N

Step 3: Set FACT = 1, I = 1

Step 4: Repeat while I <= N

 FACT = FACT * I

 I = I + 1

Step 5: Print FACT

Step 6: Stop

UNIT 5

Introduction to C Language

5.1 History and Evolution of C Language

Year	Development
1960s	ALGOL 60 introduces structured, block-based programming concepts.
1967	Martin Richards develops BCPL (Basic Combined Programming Language).
1970	Ken Thompson develops B language (a simplified version of BCPL) at Bell Labs, used to help create early UNIX.
1972	Dennis Ritchie, at Bell Laboratories, develops the C language from B, adding data types and other features — originally to rewrite the UNIX operating system.
1978	Brian Kernighan & Dennis Ritchie publish 'The C Programming Language', informally establishing the K&R C standard.
1989	The American National Standards Institute standardises the language as ANSI C (also called C89/C90).
1999	C99 standard adds new features such as inline functions and the // comment style.
2011 & later	C11 and C17/C18 add further improvements (multithreading support, better Unicode handling, etc.).

Why C matters

C is often called the 'mother of all modern programming languages' because languages such as C++, Java, C#, and even parts of Python's implementation borrow heavily from its syntax and concepts.

5.2 Characteristics of C

- Simple and Structured — programs are divided into functions/blocks, making logic easy to follow.
- Middle-Level Language — combines the features of a high-level language (readability) with those of a low-level language (direct hardware/memory access via pointers).
- Portable — a C program written on one machine/OS can be compiled and run on another with little or no change.
- Fast and Efficient — produces compact, fast executable code, close to assembly-language performance.
- Rich set of built-in Operators and Functions — supports arithmetic, relational, logical, bitwise and other operators, plus an extensive Standard Library.

- Extensible — programmers can easily add their own functions and libraries.
- Case Sensitive — treats uppercase and lowercase letters as different (e.g. Sum and sum are different identifiers).
- Pointer Support — direct memory address manipulation is possible using pointers, giving fine control over memory.

5.3 Structure of a C Program

Every C program follows a fixed general structure made up of the following sections, always in the same order:

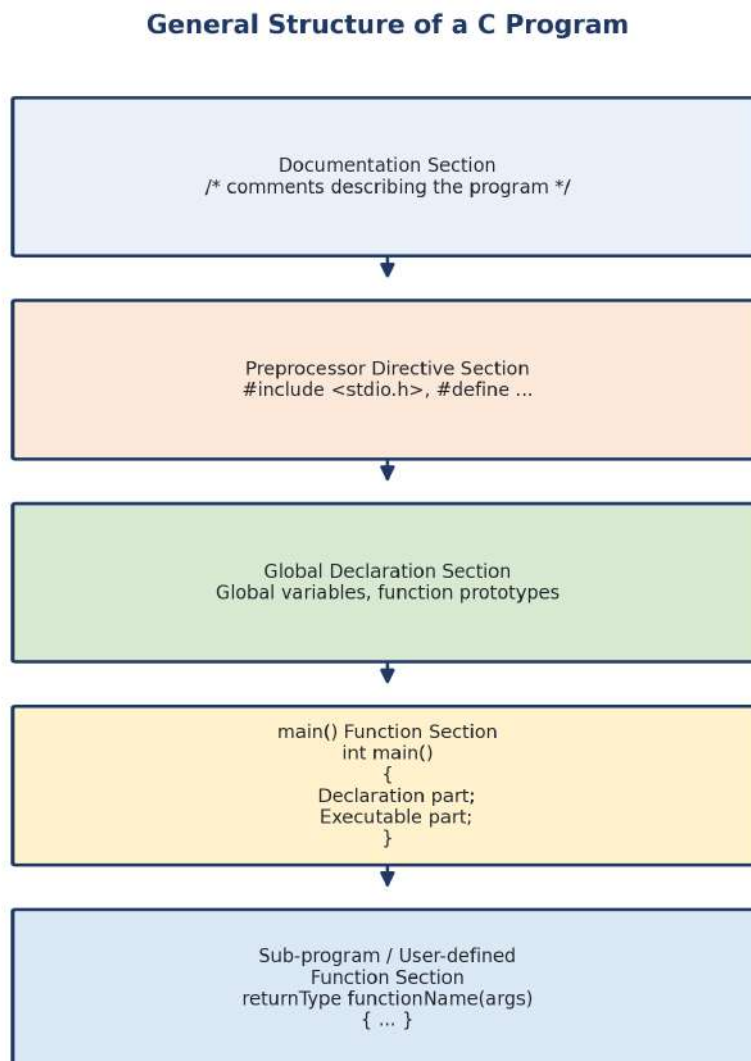


Fig. 5.1 — General structure of a C program

Section	Purpose
Documentation Section	Comments (/* ... */ or //) describing what the program does, the author's name, date, etc. Ignored by the compiler.
Preprocessor Directive Section	Statements beginning with # (e.g. #include, #define) that are processed before actual compilation.

Section	Purpose
Global Declaration Section	Global variables and function prototypes/declarations that are accessible throughout the program.
main() Function Section	Every C program must have exactly one main() function – execution always begins here. It contains a Declaration Part (variable declarations) and an Executable Part (statements).
Sub-program Section	User-defined functions, written either before or after main(), that perform specific sub-tasks and can be called from main() or from other functions.

A Minimal C Program

```

/* Documentation Section: A simple C program */
#include <stdio.h>      /* Preprocessor directive */

int main()             /* main() function - execution starts here */
{
    printf("Hello, World!\n"); /* Executable statement */
    return 0;              /* Terminates main() successfully */
}

```

OUTPUT

Hello, World!

5.4 Header Files

DEFINITION — Header File

A header file (with extension .h) is a file that contains the declarations (prototypes) of standard library functions, macros and constants. It is included in a program using the #include directive so that the compiler knows how to interpret calls to those functions.

Header File	Purpose	Example Functions
stdio.h	Standard Input/Output functions	printf(), scanf(), getchar(), putchar()
stdlib.h	Standard/general-purpose utility functions	malloc(), free(), exit(), atoi(), rand()
string.h	String handling functions	strlen(), strcpy(), strcat(), strcmp()
math.h	Mathematical functions	sqrt(), pow(), abs(), sin(), cos()
ctype.h	Character-testing/handling functions	isalpha(), isdigit(), toupper(), tolower()
conio.h	Console Input/Output (compiler-specific, e.g. Turbo C)	clrscr(), getch()

Two forms of the #include directive:

```
#include <stdio.h>    // Searches for the header in the compiler's standard sys
tem directories
#include "myheader.h" // Searches first in the current/user's project director
y
```

5.5 The C Preprocessor

DEFINITION — Preprocessor

The C preprocessor is a program that runs automatically before actual compilation begins. It scans the source code for lines beginning with '#' (preprocessor directives) and performs simple text substitution/inclusion, producing an expanded source file that is then handed to the compiler.

Directive	Purpose	Example
#include	Inserts the content of another file (usually a header file) into the program.	#include <stdio.h>
#define	Defines a macro — a symbolic name/constant or a small inline code substitution.	#define PI 3.14159
#undef	Removes/cancels a previously defined macro.	#undef PI
#ifdef / #ifndef	Conditional compilation — includes code only if a macro is/isn't defined.	#ifdef DEBUG ... #endif
#if / #else / #endif	General conditional compilation based on an expression's value.	#if VERSION > 2 ... #endif
#pragma	Gives special instructions to the compiler (implementation-specific).	#pragma warning(disable:4996)

Example Program using the Preprocessor

```
#include <stdio.h>
#define PI 3.1415    /* symbolic constant */
#define AREA(r) (PI * r * r) /* macro with an argument */

int main()
{
    float radius = 5.0, area;
    area = AREA(radius);
    printf("Area of the circle = %.2f\n", area);
    return 0;
}
```

OUTPUT

```
Area of the circle = 78.54
```

5.6 Standard Library Functions

DEFINITION — Library Function

A library function (or built-in function) is a ready-made function, already written, tested and compiled by the creators of the C compiler, and made available to the programmer through header files. The programmer only needs to '#include' the relevant header and call the function — its internal code need not be rewritten.

Example Program using several Standard Library Functions

```
#include <stdio.h>
#include <string.h>
#include <math.h>

int main()
{
    char name[20] = "Computer";
    int length;
    double result;

    length = strlen(name);          /* string.h function */
    result = sqrt(81.0);            /* math.h function */

    printf("The word is: %s\n", name);
    printf("Its length is: %d\n", length);
    printf("Square root of 81 is: %.1f\n", result);
    return 0;
}
```

OUTPUT

```
The word is: Computer
Its length is: 8
Square root of 81 is: 9.0
```

Library vs. User-defined Functions

Library functions are pre-written and provided with the compiler (e.g. printf, strlen). User-defined functions are written by the programmer for a specific task in their own program (e.g. a custom function findLargest()).

UNIT 6

Data Types and Operators

6.1 Data Types in C

DEFINITION — Data Type

A data type specifies the kind of value a variable can hold and how much memory should be reserved for it, as well as what operations can legally be performed on it.

(a) Primary / Basic Data Types

Type	Used For	Typical Size	Typical Range
int	Whole numbers (integers)	2 or 4 bytes	-2,147,483,648 to 2,147,483,647 (typical 4-byte)
float	Single-precision floating-point (decimal) numbers	4 bytes	~6 decimal digits precision
double	Double-precision floating-point numbers	8 bytes	~15 decimal digits precision
char	A single character	1 byte	-128 to 127 (or 0 to 255)
void	Represents 'no value' / an empty data type	—	Used for functions returning nothing, or generic pointers

Note: Exact sizes/ranges may vary slightly between compilers/systems; use the `sizeof()` operator to check the exact size on your own machine.

(b) Type Modifiers

Modifiers alter the size/range of the basic data types: signed, unsigned, short, long. E.g. unsigned int, long double, short int.

(c) Derived Data Types

- Array — a collection of elements of the same data type stored in contiguous memory, e.g. `int marks[10];`
- Pointer — a variable that stores the memory address of another variable, e.g. `int *ptr;`
- Function — a named, reusable block of code that performs a specific task.

(d) User-defined Data Types

- struct — groups related variables of different data types under one name.
- union — similar to struct, but all members share the same memory location.
- enum — defines a set of named integer constants for readability, e.g. `enum Day {MON, TUE, WED};`

- typedef — creates an alias/new name for an existing data type.

Example Program: Demonstrating Basic Data Types

```
#include <stdio.h>
int main()
{
    int age = 20;
    float height = 5.9;
    double distance = 384400.756;
    char grade = 'A';

    printf("Age: %d\n", age);
    printf("Height: %.1f feet\n", height);
    printf("Distance to Moon: %.3lf km\n", distance);
    printf("Grade: %c\n", grade);
    printf("Size of int = %lu bytes\n", sizeof(age));
    return 0;
}
```

OUTPUT

```
Age: 20
Height: 5.9 feet
Distance to Moon: 384400.756 km
Grade: A
Size of int = 4 bytes
```

6.2 Operators in C

DEFINITION — Operator

An operator is a symbol that instructs the compiler to perform a specific mathematical, relational or logical operation on one or more operands (values/variables).

(a) Arithmetic Operators

Operator	Meaning	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus (remainder after integer division)	a % b

(b) Relational Operators

Operator	Meaning	Example
==	Equal to	a == b

Operator	Meaning	Example
!=	Not equal to	a != b
>	Greater than	a > b
<	Less than	a < b
>=	Greater than or equal to	a >= b
<=	Less than or equal to	a <= b

(c) Logical Operators

Operator	Meaning	Example
&&	Logical AND – true only if both conditions are true	(a>0 && b>0)
	Logical OR – true if at least one condition is true	(a>0 b>0)
!	Logical NOT – reverses the result of a condition	!(a>0)

(d) Assignment Operators

Operator	Equivalent To	Example
=	Assigns the right-hand value to the left-hand variable	a = 5
+=	a = a + b	a += 5
-=	a = a - b	a -= 5
*=	a = a * b	a *= 5
/=	a = a / b	a /= 5
%=	a = a % b	a %= 5

(e) Increment / Decrement Operators

- ++ Increment – increases the value of a variable by 1 (e.g. a++ or ++a).
- -- Decrement – decreases the value of a variable by 1 (e.g. a-- or --a).
- Pre (++a) – value is changed before it is used in the expression.
- Post (a++) – value is used in the expression first, then changed.

(f) Other Operators

Category	Symbol(s)	Description
Bitwise	&, , ^, ~, <<, >>	Operate directly on the individual bits of integer data.
Conditional (Ternary)	condition ? expr1 : expr2	A compact one-line if-else; evaluates expr1 if condition is true, else expr2.

Category	Symbol(s)	Description
Comma	,	Separates multiple expressions to be evaluated in sequence, e.g. in a for loop.
sizeof	sizeof(x)	Returns the size (in bytes) of a variable or data type.

Example Program: Demonstrating Operators

```
#include <stdio.h>
int main()
{
    int a = 15, b = 4;

    printf("a + b = %d\n", a + b);
    printf("a - b = %d\n", a - b);
    printf("a * b = %d\n", a * b);
    printf("a / b = %d\n", a / b);      /* integer division */
    printf("a %% b = %d\n", a % b);    /* modulus (remainder) */

    printf("Is a > b? %d\n", a > b);   /* relational: 1 = true */
    printf("a && b : %d\n", a && b);    /* logical AND */

    int larger = (a > b) ? a : b;      /* ternary/conditional operator */
    printf("Larger value = %d\n", larger);

    return 0;
}
```

OUTPUT

```
a + b = 19
a - b = 11
a * b = 60
a / b = 3
a % b = 3
Is a > b? 1
a && b : 1
Larger value = 15
```

(g) Operator Precedence (Quick Reference, high to low)

Precedence	Operators	Category
1 (Highest)	() [] -> .	Parentheses, array subscript, member access
2	! ~ ++ -- (type) sizeof	Unary operators
3	* / %	Multiplicative
4	+ -	Additive
5	<< >>	Bitwise shift
6	< <= > >=	Relational
7	== !=	Equality

Precedence	Operators	Category
8	&&	Logical AND
9		Logical OR
10	?:	Ternary (conditional)
11 (Lowest)	= += -= *= /= ...	Assignment

Remember

When operators of the same precedence appear together, associativity decides the order. most operators associate left to right, but assignment and unary operators associate right to left.

End of Notes

Revise each definition box, re-draw the diagrams from memory, and trace through every example program by hand to fix the logic firmly in mind before an exam.